

Documentazione Test

1. Test di Unità

Package: `model`

Classe Testata: `NodeDT0`

Test 1 (`testInternalNodeCreation`)

Significato: verifica che, alla creazione di un nodo interno di split, lo stato venga inizializzato correttamente: il nodo non deve essere una foglia, deve mantenere la condizione di split passata, non deve avere un valore predetto e deve partire senza figli.

Campo	Contenuto
Input	<code>splitCondition = "X <= 10"</code>
Azione	<code>new NodeDT0(splitCondition)</code>
Atteso	<code>leaf = false</code> ; <code>splitCondition = "X <= 10"</code> ; <code>predictedValue = null</code> ; <code>children</code> inizializzata come lista vuota

Test 2 (`testLeafNodeCreation`)

Significato: verifica che, alla creazione di un nodo foglia tramite il costruttore numerico, il nodo risulti correttamente marcato come foglia, con il valore stimato coerente e nessuna condizione di split.

Campo	Contenuto
Input	<code>predictedValue = 42.5</code>
Azione	<code>new NodeDT0(predictedValue)</code> (costruttore numerico)
Atteso	<code>leaf = true</code> ; <code>predictedValue = 42.5</code> ; <code>splitCondition = null</code>

Test 3 (`testAddChild`)

Significato: verifica che l'inserimento di un nodo figlio in un nodo interno funzioni correttamente, aggiornando la lista dei figli con il riferimento corretto.

Campo	Contenuto
Input	nodo interno <code>n</code> con <code>children</code> vuota; nodo figlio <code>m</code> da inserire
Azione	<code>n.addChild(m)</code>
Atteso	<code>n.getChildren().size() == 1</code> e <code>children</code> contiene esattamente il riferimento a <code>m</code>

Test 4 (`testMultipleChildren`)

Significato: verifica che, inserendo più nodi figli in sequenza, l'ordine di inserimento venga mantenuto inalterato nella lista dei figli.

Campo	Contenuto
Input	nodo padre <code>n</code> ; sequenza ordinata di nodi figli <code>m1, m2, ..., mk</code>
Azione	inserimento sequenziale: <code>n.addChild(m1)</code> , <code>n.addChild(m2)</code> ,..., <code>n.addChild(mk)</code>
Atteso	<code>n.getChildren()</code> restituisce la lista <code>[m1, m2, ..., mk]</code> nello stesso ordine di inserimento

Test 5 (`testDefaultSelected`)

Significato: verifica lo stato predefinito del flag di selezione visiva, che deve garantire che un nodo appena creato sia visibile durante il rendering grafico.

Campo	Contenuto
Input	nessuno (nuova istanza)
Azione	<code>new NodeDTO(...)</code>
Atteso	<code>selected = true</code> (valore predefinito)

Test 6 (`testToStringInternalNode`)

Significato: verifica che la rappresentazione testuale di un nodo interno coincida esattamente con la condizione di split memorizzata, così da poter essere usata direttamente nell'interfaccia grafica.

Campo	Contenuto
Input	nodo interno <code>n</code> con <code>splitCondition</code> valorizzata
Azione	<code>n.toString()</code>
Atteso	stringa restituita <code>== n.splitCondition</code>

Test 7 (`testToStringLeafNode`)

Significato: verifica che la rappresentazione testuale di un nodo foglia mostri in modo leggibile il valore predetto, con un prefisso descrittivo standard.

Campo	Contenuto
Input	nodo foglia <code>n</code> con <code>predictedValue</code> valorizzato
Azione	<code>n.toString()</code>
Atteso	stringa restituita nel formato <code>"Predizione: <valore>"</code> , con <code><valore></code> pari a <code>predictedValue</code>

Test 8 (`testAddChildToLeafNode`)

Significato: verifica il comportamento del sistema quando si tenta di aggiungere un ramo a un nodo che è una foglia.

Campo	Contenuto
Input	nodo <code>n</code> istanziato come foglia (<code>leaf = true</code>)
Azione	<code>n.addChild(m)</code>
Atteso	<code>n.leaf</code> diventa <code>false</code>

Test 9 (`testToStringWithNullValues`)

Significato: verifica la robustezza del metodo `toString()` quando i campi rilevanti del nodo non sono valorizzati, assicurando che l'assenza di dati non generi errori a runtime.

Campo	Contenuto
Input	nodo <code>n</code> con <code>splitCondition = null</code> oppure <code>predictedValue = null</code>
Azione	<code>n.toString()</code>
Atteso	il metodo gestisce l'assenza di dati senza sollevare <code>NullPointerException</code>

Classe Testata: `TreeDT0`

Test 1 (`testConstructor`)

Significato: verifica che l'aggregazione iniziale tra la radice dell'albero e il nome della tabella di training avvenga correttamente al momento della costruzione dell'oggetto.

Campo	Contenuto
Input	<code>root</code> (istanza di <code>NodeDT0</code>); <code>tableName</code> (nome della tabella)
Azione	<code>new TreeDT0(root, tableName)</code>
Atteso	<code>getRoot() == root; getTableName() == tableName</code>

Package: `view`

Classe Testata: `ControlPanel`

Test 1 (`testDefaultValues`)

Significato: verifica che, all'avvio, i controlli grafici del pannello siano preimpostati con valori di default coerenti e che il pulsante di predizione sia disabilitato finché non sono soddisfatte le condizioni necessarie.

Campo	Contenuto
Input	nessuno (pannello appena istanziato)
Azione	<code>new ControlPanel()</code>

Atteso	<code>getServerAddress() == "127.0.0.1"; getServerPort() == 8080;</code> <code>getTableName() == "provac"; getPredictButton().isEnabled() == false</code>
--------	--

Test 2 (`testGetServerPort`)

Significato: verifica che il valore testuale inserito nel campo porta venga correttamente convertito in un intero utilizzabile dal controller.

Campo	Contenuto
Input	campo porta valorizzato con una stringa numerica valida (es. <code>"8080"</code>)
Azione	<code>cp.getServerPort()</code>
Atteso	valore restituito <code>8080</code> (intero primitivo)

Test 3 (`testGetServerPortInvalid`)

Significato: verifica la reazione del sistema quando l'utente inserisce caratteri non numerici nel campo porta, in modo che il controller possa intercettare l'errore di digitazione.

Campo	Contenuto
Input	campo porta valorizzato con caratteri alfabetici
Azione	<code>cp.getServerPort()</code>
Atteso	viene sollevata <code>NumberFormatException</code>

Test 4 (`testIsDatabaseSource`)

Significato: verifica che la selezione della sorgente dati nel combo box sia correttamente riflessa dal metodo che discrimina tra origine database e origine archivio.

Campo	Contenuto
Input	<code>sourceComboBox</code> impostato alternativamente sull'indice <code>0</code> (<code>"Da Database"</code>) e sull'indice <code>1</code> (<code>"Da Archivio (.dmp)"</code>)
Azione	<code>cp.isDatabaseSource()</code>

Atteso	restituisce <code>true</code> quando l'indice selezionato è <code>0</code> ; restituisce <code>false</code> quando l'indice selezionato è <code>1</code>
--------	--

Test 5 (`testGetServerPortWithWhitespace`)

Significato: verifica la tolleranza del parsing della porta rispetto a spazi bianchi accidentali inseriti dall'utente prima o dopo il valore numerico.

Campo	Contenuto
Input	campo porta valorizzato con stringa contenente spazi attorno alle cifre (es. <code>" 9090 "</code>)
Azione	<code>cp.getServerPort()</code> (implementato come <code>Integer.parseInt(portField.getText().trim())</code>)
Atteso	il valore viene sanitizzato tramite <code>trim()</code> prima del parsing e restituisce <code>9090</code> ; se il campo contiene solo spazi, <code>trim()</code> produce una stringa vuota e viene sollevata <code>NumberFormatException</code>

Classe Testata: `SummaryPanel`

Test 1 (`testSetPredictionWithValue`)

Significato: verifica che il valore predetto venga mostrato all'utente con una formattazione numerica ad alta precisione (4 cifre decimali), per garantire leggibilità e coerenza nella visualizzazione dei risultati.

Campo	Contenuto
Input	<code>value = 3.14</code>
Azione	<code>sp.setPrediction(value)</code>
Atteso	l'etichetta mostra il testo <code>"3.1400"</code>

Test 2 (`testSetPredictionWithNull`)

Significato: verifica che, in assenza di una predizione disponibile, l'interfaccia mostri un segnaposto chiaro invece di un valore vuoto o un errore.

Campo	Contenuto
Input	<code>value = null</code>
Azione	<code>sp.setPrediction(null)</code>
Atteso	l'etichetta mostra il testo segnaposto predefinito <code>" - - - "</code>

Test 3 (`testReset`)

Significato: verifica che il ripristino del pannello laterale riporti tutti i parametri di sessione (contatore passi, valore stimato) al loro stato iniziale.

Campo	Contenuto
Input	pannello con valori arbitrari già impostati (contatore passi > 0, etichetta valorizzata)
Azione	<code>sp.reset()</code>
Atteso	contatore dei passi azzerato; etichetta del valore stimato torna a <code>" - - - "</code>

Test 4 (`testAddHistoryEntry`)

Significato: verifica che ogni predizione effettuata venga correttamente aggiunta alla cronologia visibile all'utente, con il percorso decisionale e l'esito numerico formattati.

Campo	Contenuto
Input	percorso decisionale (es. sequenza di condizioni attraversate); esito numerico della predizione
Azione	<code>sp.addHistoryEntry(percorso, esito)</code>
Atteso	il <code>DefaultListModel</code> sottostante contiene un nuovo elemento con il testo formattato atteso

Classe Testata: `LogPanel`

Test 1 (`testLogInfoMessage`)

Significato: verifica che un messaggio privo di parole chiave particolari venga classificato di default come informativo, comportamento base del sistema di log.

Campo	Contenuto
Input	messaggio privo di termini critici (es. "Operazione avviata")
Azione	<code>lp.append(messaggio)</code>
Atteso	il documento di log applica il formato di default <code>INFO</code>

Test 2 (`testLogErrorMessage`)

Significato: verifica che i messaggi contenenti indicazioni di errore vengano riconosciuti e classificati correttamente come `ERROR` , per attirare l'attenzione dell'utente.

Campo	Contenuto
Input	messaggio contenente la parola "errore"
Azione	<code>lp.append(messaggio)</code>
Atteso	il messaggio viene classificato con livello <code>ERROR</code>

Test 3 (`testLogSuccessMessage`)

Significato: verifica che i messaggi che segnalano un esito positivo vengano correttamente classificati come `SUCCESS` .

Campo	Contenuto
Input	messaggio contenente la parola "successo"
Azione	<code>lp.append(messaggio)</code>
Atteso	il log assegna il livello <code>SUCCESS</code>

Test 4 (`testLogWarningMessage`)

Significato: verifica che i messaggi di avviso vengano correttamente classificati come `WARN` , distinguendoli dai messaggi puramente informativi o di errore.

Campo	Contenuto
-------	-----------

Input	messaggio contenente la parola <code>"warning"</code>
Azione	<code>lp.append(messaggio)</code>
Atteso	la riga riceve la categorizzazione <code>WARN</code>

Test 5 (`testClear`)

Significato: verifica che la funzione di pulizia della console di log svuoti effettivamente il contenuto testuale accumulato, senza lasciare residui.

Campo	Contenuto
Input	pannello di log con diverse righe già scritte
Azione	<code>lp.clear()</code>
Atteso	la lunghezza del testo nel documento torna a <code>0</code> caratteri

Test 6 (`testLogMessageStylesApplied`)

Significato: verifica che, oltre alla classificazione logica del messaggio, vengano effettivamente applicati gli stili grafici (colori) corrispondenti nel documento visualizzato, così da garantire una reale distinzione visiva tra i tipi di messaggio.

Campo	Contenuto
Input	un messaggio di errore e un messaggio di successo inviati al <code>LogPanel</code>
Azione	analisi degli attributi grafici (<code>AttributeSet</code>) estratti dallo <code>StyledDocument</code> per ciascun messaggio
Atteso	i caratteri dei due messaggi presentano colori di primo piano (<code>foreground</code>) distinti e conformi alle costanti cromatiche definite per <code>ERROR</code> e <code>SUCCESS</code>

Classe Testata: `TreePanel`

Test 1 (`testDefaultState`)

Significato: verifica lo stato iniziale dell'area di rendering grafico, assicurando che non venga disegnato nulla finché non è stato fornito un albero valido.

Campo	Contenuto
Input	nessuno (pannello appena istanziato)
Azione	<code>new TreePanel()</code>
Atteso	<code>currentTree == null</code>

Test 2 (`testUpdateTree`)

Significato: verifica che l'assegnazione di una nuova struttura ad albero venga correttamente memorizzata dal pannello, come base per il successivo calcolo delle coordinate di disegno.

Campo	Contenuto
Input	istanza di <code>TreeDT0</code> da visualizzare
Azione	<code>tp.updateTree(treeDT0)</code>
Atteso	il pannello memorizza <code>treeDT0</code> come nuova radice corrente (<code>currentTree == treeDT0</code>)

Test 3 (`testResetTree`)

Significato: verifica che l'azzeramento dell'area di disegno rimuova correttamente l'albero attualmente visualizzato, riportando il pannello allo stato vuoto.

Campo	Contenuto
Input	pannello con un albero già assegnato e visualizzato
Azione	<code>tp.resetTree()</code>
Atteso	<code>currentTree</code> torna a <code>null</code>

Package: `network`

Classe Testata: `ServerConnection`

Test 1 (`testInitialNotConnected`)

Significato: verifica che, appena creata, un'istanza di `ServerConnection` non risulti connessa, prima di qualsiasi tentativo di comunicazione di rete.

Campo	Contenuto
Input	nessuno (istanza appena creata)
Azione	<code>sc.isConnected()</code>
Atteso	restituisce <code>false</code>

Test 2 (`testConnect`)

Significato: verifica che l'instaurazione del canale di comunicazione via socket avvenga correttamente, con lo stato di connessione aggiornato di conseguenza.

Campo	Contenuto
Input	<code>ServerSocket</code> locale in ascolto su una porta nota
Azione	<code>sc.connect(host, port)</code>
Atteso	il socket si apre correttamente; <code>sc.isConnected() == true</code>

Test 3 (`testClose`)

Significato: verifica il corretto rilascio delle risorse di rete e degli stream associati quando la connessione viene chiusa esplicitamente.

Campo	Contenuto
Input	connessione attiva (<code>sc.isConnected() == true</code>)
Azione	<code>sc.close()</code>
Atteso	il socket viene chiuso; <code>sc.isConnected() == false</code>

Test 4 (`testDoubleClose`)

Significato: verifica l'idempotenza dell'operazione di chiusura, così che invocarla più volte non causi errori o eccezioni impreviste.

Campo	Contenuto
-------	-----------

Input	connessione già chiusa in precedenza
Azione	<code>sc.close()</code> invocato una seconda volta consecutiva
Atteso	nessuna eccezione anomala viene sollevata

Test 5 (`testSendWithoutConnection`)

Significato: verifica che il sistema impedisca (con un blocco difensivo) l'invio di dati quando il canale di comunicazione non è attivo.

Campo	Contenuto
Input	<code>ServerConnection</code> senza socket aperto
Azione	<code>sc.send(dato)</code>
Atteso	viene sollevata <code>IOException</code>

Test 6 (`testReceiveWithoutConnection`)

Significato: verifica che il sistema impedisca (con un blocco difensivo) la ricezione di dati quando non esiste una connessione attiva.

Campo	Contenuto
Input	<code>ServerConnection</code> senza connessione attiva
Azione	<code>sc.receive()</code>
Atteso	il tentativo di lettura fallisce sollevando <code>IOException</code>

Test 7 (`testSendAndReceive`)

Significato: verifica lo scambio effettivo e corretto di oggetti serializzati attraverso il socket, garantendo l'integrità del dato inviato e ricevuto.

Campo	Contenuto
Input	server mock locale attivo; oggetto stringa da inviare
Azione	<code>sc.send(oggetto)</code> seguito da <code>sc.receive()</code> (con il server mock che restituisce un eco)

Atteso	l'oggetto ricevuto è integro e coincide con quello inviato (echo)
--------	---

Test 8 (`testConnectClosesPrevious`)

Significato: verifica che, in caso di riconnessione verso un nuovo indirizzo/porta, il canale precedente venga chiuso implicitamente prima di aprirne uno nuovo, evitando connessioni orfane.

Campo	Contenuto
Input	connessione già attiva verso una prima porta; nuova richiesta di connessione verso una porta differente
Azione	<code>sc.connect(host, nuovaPorta)</code> (seconda invocazione)
Atteso	il socket precedente viene chiuso prima che il nuovo venga inizializzato

Classe Testata: `PredictionWorker`

Test 1 (`testParseBranchOptionsWithNumbers`)

Significato: verifica che il parser rimuova correttamente i prefissi numerici dalle opzioni di risposta ricevute dal server, restituendo solo il testo pulito delle opzioni.

Campo	Contenuto
Input	stringa con opzioni numerate, es. <code>"0: Si\n1: No"</code>
Azione	<code>pw.parseBranchOptions(stringa)</code> (metodo privato di <code>PredictionWorker</code> , invocato tramite reflection)
Atteso	restituisce la lista <code>["Si", "No"]</code> , con i prefissi numerici (<code>"0:"</code> , <code>"1:"</code>) rimossi tramite <code>substring</code> / <code>trim</code>

Test 2 (`testParseBranchOptionsWithoutNumbers`)

Significato: verifica che il parser gestisca correttamente anche il caso in cui le opzioni testuali siano prive di indici numerici, conservandole senza alterazioni.

Campo	Contenuto
-------	-----------

Input	stringhe su righe separate (<code>\n</code>), nessuna delle quali inizia con il pattern <code>^[0-9]+:.*</code>
Azione	<code>pw.parseBranchOptions(stringa)</code>
Atteso	le stringhe vengono restituite invariate (solo trim degli spazi ai bordi)

Test 3 (`testParseBranchOptionsWithEmptyLines`)

Significato: verifica che il parser gestisca correttamente righe vuote o composte solo da spazi all'interno del messaggio ricevuto dal server, scartandole dal risultato finale.

Campo	Contenuto
Input	risposta contenente salti riga consecutivi (righe vuote o composte solo da spazi)
Azione	<code>pw.parseBranchOptions(stringa)</code>
Atteso	le righe che, dopo il <code>trim()</code> , risultano vuote (<code>opt.isEmpty()</code>) vengono escluse dalla lista restituita

Classe Testata: `TableLoaderWorker`

Test 1 (`testTableLoaderWorkerExecution`)

Significato: verifica che l'interrogazione asincrona dell'elenco delle tabelle disponibili sul server avvenga correttamente e senza deadlock, popolando il modello dati usato dall'interfaccia utente.

Campo	Contenuto
Input	<code>ServerSocket</code> simulato che risponde al comando <code>4</code> con una lista di tabelle; <code>CountDownLatch</code> per sincronizzare il test
Azione	avvio del <code>TableLoaderWorker</code> in background e attesa tramite latch, quindi <code>worker.get()</code>
Atteso	<code>get()</code> restituisce l'elenco atteso delle tabelle, senza deadlock né blocchi del thread

Classe Testata: `InitTreeWorker`

Test 1 (`testInitTreeWorkerExecution`)

Significato: verifica che il caricamento (o apprendimento) asincrono dell'albero dal server avvenga correttamente, restituendo l'esito atteso al termine dell'elaborazione in background.

Campo	Contenuto
Input	comando <code>0</code> con nome tabella, inviato verso un server mock che risponde <code>"OK"</code>
Azione	avvio dell' <code>InitTreeWorker</code> , attesa tramite latch fino al completamento
Atteso	l'esito restituito dal thread di background coincide con la stringa di conferma <code>"OK"</code>

Package: `controller`

Classe Testata: `MainController`

Test 1 (`testConstructorCreatesConnection`)

Significato: verifica che, alla creazione del controller principale, vengano correttamente istanziati sia il gestore della connessione di rete sia i collegamenti (listener) tra la vista e la logica applicativa.

Campo	Contenuto
Input	istanza di <code>MainFrame</code>
Azione	<code>new MainController(mainFrame)</code> (il costruttore chiama internamente <code>initListeners()</code>)
Atteso	viene creata un'istanza interna di <code>ServerConnection</code> ; <code>getLoadTreeButton()</code> , <code>getPredictButton()</code> e <code>getRefreshTablesButton()</code> ricevono ciascuno il proprio <code>ActionListener</code> , e alla finestra viene agganciato un <code>WindowListener</code> che chiude la connessione in <code>windowClosing</code>

Test 2 (`testPredictButtonDisabledWithoutConnection`)

Significato: verifica che il controller impedisca l'avvio di una predizione quando non esiste una connessione di rete attiva, avvisando l'utente in modo chiaro.

Campo	Contenuto
Input	controller con <code>connection.isConnected() == false</code>
Azione	<code>mc.handlePredict()</code> (metodo privato, invocato tramite reflection)
Atteso	il metodo termina senza istanziare/avviare un <code>PredictionWorker</code> ; viene mostrato un <code>JOptionPane</code> di tipo <code>WARNING_MESSAGE</code> con testo "Nessuna connessione attiva. Inizializzare prima l'albero."

Test 3 (`testEmptyTableNameShowsWarning`)

Significato: verifica che il controller validi il nome della tabella prima di avviare l'inizializzazione dell'albero, bloccando la richiesta se il campo è vuoto.

Campo	Contenuto
Input	<code>getTableName()</code> restituisce <code>null</code> oppure <code>""</code>
Azione	<code>mc.handleInitTree()</code> (metodo privato, invocato tramite reflection)
Atteso	il metodo termina prima di istanziare <code>InitTreeWorker</code> ; viene mostrato un <code>JOptionPane</code> di tipo <code>WARNING_MESSAGE</code> con testo "Inserire il nome della tabella o del file di archivio!"

Test 4 (`testInvalidPortShowsError`)

Significato: verifica che un errore di formato nel campo porta venga correttamente intercettato e segnalato graficamente all'utente durante il caricamento dell'elenco delle tabelle dal database.

Campo	Contenuto
Input	<code>getServerPort()</code> solleva <code>NumberFormatException</code> (caratteri non numerici nel campo porta)
Azione	<code>mc.handleRefreshTables()</code> (metodo privato collegato all'azione di <code>refreshTablesButton</code> , invocato tramite reflection)

Atteso	l'eccezione viene intercettata dal blocco <code>try/catch</code> ; <code>TableLoaderWorker</code> non viene avviato; viene mostrato un <code>JOptionPane</code> di tipo <code>ERROR_MESSAGE</code> con testo "Porta non valida."
--------	--

Test 5 (`testHandleInitTreeSourceArchive`)

Significato: verifica che, selezionando l'origine dati "Da Archivio", il controller indirizzi correttamente la richiesta verso il caricamento da file serializzato, evitando chiamate al database relazionale.

Campo	Contenuto
Input	<code>isDatabaseSource() == false</code> (sourceComboBox su "Da Archivio (.dmp)", indice <code>1</code>); nome tabella/file e porta validi
Azione	<code>mc.handleInitTree()</code> (metodo privato, invocato tramite reflection)
Atteso	viene istanziato <code>InitTreeWorker</code> con il parametro <code>fromDB = false</code> , disattivando così le chiamate al database relazionale in favore della lettura da archivio